
RESOBOT

Enterprise Incident Management System

A complete developer guide and hand-off manual for installation, configuration, and operations of the automated data ingestion and AI routing platform.

1. System Architecture Overview

ResoBot is a full-stack, enterprise-grade Incident Management platform built on the MERN stack. It leverages advanced integrations to automate workflows, parse documents intelligently, and maintain a secure, real-time workspace for administration.

- **Frontend:** React.js (Vite), Tailwind CSS, React Router, Context API.
- **Backend:** Node.js, Express.js, Socket.io for WebSocket real-time updates.
- **Database:** MongoDB (Mongoose ODM).
- **AI Engine:** OpenAI API (GPT-4o) for OCR, classification, and summarization.
- **Automation:** UiPath Orchestrator API for headless bot triggering.
- **Security:** JWT Auth, Bcrypt Password Hashing, Helmet.js, Express Rate Limiting.

2. System Prerequisites

Before installing the codebase on a new machine, ensure the following software and accounts are available:

- **Node.js:** Version 18.x or higher installed on the host machine.
- **MongoDB:** A local MongoDB instance or a cloud MongoDB Atlas URI.
- **OpenAI Account:** An API key with access to the `gpt-4o` model.
- **UiPath Cloud Account:** Valid Client ID, Secret, and Orchestrator Folder details.

- **Email SMTP:** An App Password from a provider (e.g., Gmail) to send notification alerts.

Mac OS / iCloud Warning: Never place this project folder inside your macOS "Documents" or "Desktop" folders if iCloud Sync is enabled. Doing so will cause severe `ETIMEDOUT` errors during dependency installation and server boot. Move the project to a dedicated `~/Projects` directory in your user root.

3. Environment Configuration (.env)

The system requires environment variables to securely connect to third-party services. You must create two separate `.env` files.

Backend Environment

Create a file named `.env` inside the `/backend` directory:

```
# Server Configuration
PORT=5001
NODE_ENV=development
FRONTEND_URL=http://localhost:5173

# Database & Security
MONGO_URI=mongodb+srv://:@cluster.mongodb.net/resobot
JWT_SECRET=generate_a_random_secure_string_here

# AI Integration
OPENAI_API_KEY=sk-your-openai-api-key

# Email Alerts (Nodemailer config)
EMAIL_USER=your_corporate_email@gmail.com
EMAIL_PASS=your_16_digit_app_password

# UiPath Orchestrator Integration
UIPATH_CLIENT_ID=your_uipath_client_id
UIPATH_CLIENT_SECRET=your_uipath_secret
UIPATH_ACCOUNT=your_account_name
UIPATH_TENANT=your_tenant_name
UIPATH_FOLDER_ID=your_folder_id
UIPATH_PROCESS_NAME=Your_Release_Name
```

Frontend Environment

Create a file named `.env` inside the `/frontend` directory:

```
VITE_API_URL=http://localhost:5001/api
```

4. Installation & First Boot

To initialize the application, you must install dependencies for both the frontend and backend architectures independently.

Step 1: Install Dependencies

Open two separate terminal windows.

```
# Terminal 1: Backend
cd backend
npm install

# Terminal 2: Frontend
cd frontend
npm install
```

Step 2: Start the Development Servers

Run the following command in both terminal windows:

```
npm run dev
```

The Auto-Seed Feature: Upon the first successful connection to MongoDB, the backend server will automatically generate a Master Admin account. Look for this message in the backend terminal:

✓ Database reset: Fresh Admin auto-seeded successfully!

5. Logging In & System Usage

Once both servers are running, navigate to `http://localhost:5173` in your web browser.

Authentication

Use the auto-seeded credentials to access the secure workspace:

- **Email:** admin@dhl.com
- **Password:** password123

Core Modules Overview

Module	Description & Usage
Dashboard	A high-level statistical overview of incident resolutions, SLA compliance, and system status. Powered by real-time WebSocket events.
Incident Vault	The primary data grid for all recorded incidents. Users can click into specific incidents to update resolution statuses and leave audit-trail comments.
Upload & Draft	Manual entry portal. Drag and drop a document or image to have OpenAI instantly parse, classify, and draft a structured incident report before committing it to the database.
Bot Control	The Orchestrator bridge. Allows the admin to force-start the UiPath Bot in the cloud. The bot will scan Google Drive and push new incidents to the Node.js API webhook automatically.
Team Directory	User management panel. Create new employee accounts, assign initial secure passwords, and define system roles (Admin, Manager, Agent). Passwords are automatically hashed via bcrypt.

6. Developer Notes & Architecture Decisions

Bcrypt & Mongoose Pre-Save Hooks

The `Employee.js` model uses a modern asynchronous pre-save hook to hash passwords automatically. This ensures that passwords are never stored in plain text. When updating user profiles, the `employeeController` uses `.save()` rather than `findByIdAndUpdate` to ensure this hashing middleware is not bypassed.

Rolldown / Vite Warnings

During frontend builds, you may see a warning stating: `[PLUGIN_TIMINGS] Your build spent significant time in plugin `externalize-deps``. This is a benign warning from the Vite/Rolldown bundler regarding dependency scanning. It does not affect application performance or code quality and can be safely ignored during development.

JWT Security

The platform issues JSON Web Tokens (JWT) upon successful login, valid for 30 days. The React `AuthContext` manages this token in local storage, providing persistent login states across page reloads while maintaining route protection via the `ProtectedRoute` wrapper in `App.jsx`.